
Shell Scripting Fundamentals

Study Material

CS3101: Operating System Lab



Contents

1 Purpose and Learning Outcomes	1
2 Bash Script Fundamentals	1
2.1 Shell and shell script	1
2.2 The shebang line	1
2.3 Creating and executing a script	2
2.4 Comments	2
3 Variables, Expansion, and Quoting	2
3.1 Variable assignment	2
3.2 Variable expansion	2
3.3 Why double quotes matter	3
4 Reading User Input	3
5 Conditional Statements and Test Expressions	3
5.1 if-elif-else syntax	3
5.2 Integer comparison operators	4
5.3 Number classification	4
6 Arithmetic and Looping	4
6.1 Arithmetic expansion	4
6.2 for loop	4
6.3 Even and odd test	5
6.4 while loop	5
7 Input Validation with Bash Regular Expressions	5
7.1 Decimal interpretation of leading zeros	6
8 Palindrome Logic	6
9 Command-Line Arguments and Positional Parameters	7
10 File-Test Operators	7
10.1 Step-by-step path validation	8
11 Reading a File One Line at a Time	8
12 Command Substitution	8

13 The case Statement	9
13.1 General syntax	9
13.2 Menu dispatch	9
14 Standard Output, Standard Error, and Exit Status	10
15 Complete Corrected Program 1: Number Classification and Loop Processing	11
16 Complete Corrected Program 2: Palindrome Number Checker	12
17 Complete Corrected Program 3: File Line Counter	13
18 Complete Corrected Program 4: System Information Menu	14
19 Compact Syntax Reference	14
20 Common Errors and Their Corrections	15
21 Recommended Testing	15
22 Summary	15

1 Purpose and Learning Outcomes

This material explains the Bash concepts used in four introductory programs:

1. accepting a number, classifying it, and processing a loop;
2. reversing a non-negative integer and checking whether it is a palindrome;
3. validating a file path and counting the lines in a readable file; and
4. presenting a menu with a `case` statement.

After studying the examples, a student should be able to:

- create and execute a Bash script;
- declare variables and safely read user input;
- use `if`, `elif`, `else`, `for`, `while`, and `case`;
- perform integer arithmetic using arithmetic expansion;
- validate numeric input using a regular expression;
- use positional parameters and file-test operators;
- read a text file safely, one line at a time;
- use command substitution to capture command output; and
- report errors through standard error and return meaningful exit codes.

Important: The four examples are four separate shell scripts. Do not paste them into one executable file. A shell program stops when it reaches `exit`, so put each program in its own `.sh` file.

2 Bash Script Fundamentals

2.1 Shell and shell script

Shell. A command interpreter that accepts commands from a user or from a file and asks the operating system to execute them. Bash stands for *Bourne Again Shell*.

Shell script. A plain-text file containing shell commands that are executed in sequence. Shell scripts commonly use the filename extension `.sh`, although the extension is not compulsory.

2.2 The shebang line

The first line selects the interpreter that should run the script.

```
#!/usr/bin/env bash
```

`#!/usr/bin/env bash` searches the current environment for the `bash` executable. It is generally more portable than assuming Bash is always stored at `/bin/bash`.

2.3 Creating and executing a script

```
nano program.sh
chmod +x program.sh
./program.sh
```

- `nano program.sh` creates or edits the file.
- `chmod +x program.sh` adds execute permission.
- `./program.sh` executes the file from the current directory.

Alternatively, a script can be passed directly to Bash:

```
bash program.sh
```

2.4 Comments

A comment begins with `#`. Bash ignores the remaining text on that line.

```
# This is a comment.
sum=0 # This is an end-of-line comment.
```

Comments should explain *why* a step exists, rather than repeat an obvious command.

3 Variables, Expansion, and Quoting

3.1 Variable assignment

Variable. A named location used by the shell to store text or a value. Bash variables do not require an explicit data type declaration.

```
number_1=100
sum=0
type="Even"
```

There must be no spaces around the assignment operator. Therefore, `sum=0` is correct, while `sum = 0` is interpreted as a command and will fail.

3.2 Variable expansion

A dollar sign asks Bash to replace a variable name with its stored value.

```
echo "The number is $number_1"
echo "The final sum is: $sum"
```

Braces can clearly mark the boundary of a variable name:

```
echo "Result: ${sum}"
```

3.3 Why double quotes matter

Double quotes preserve a value as one shell argument and prevent unexpected word splitting and pathname expansion.

```
if [ "$number" -gt 0 ]; then
    echo "Positive"
fi
```

As a safe general rule, quote variable expansions unless a particular Bash operation explicitly requires otherwise.

4 Reading User Input

read. A Bash built-in command that reads one line of input and stores it in one or more variables.

```
read -r -p "Enter a number: " number
```

Part	Meaning
<code>read</code>	invokes the Bash input command
<code>-r</code>	prevents backslashes from being treated as escape characters
<code>-p</code>	displays a prompt before accepting input
<code>"Enter a number: "</code>	prompt presented to the user
<code>number</code>	variable that receives the entered text

The value returned by `read` is initially text. Validate it before using a numeric comparison or arithmetic operation.

5 Conditional Statements and Test Expressions

5.1 if-elif-else syntax

```
if [ condition_1 ]; then
    commands
elif [ condition_2 ]; then
    commands
else
    commands
fi
```

`then` begins the commands associated with a true condition, and `fi` ends the complete conditional statement.

5.2 Integer comparison operators

Operator	Meaning	Example
-eq	equal to	["\$a" -eq "\$b"]
-ne	not equal to	["\$a" -ne "\$b"]
-gt	greater than	["\$a" -gt 0]
-ge	greater than or equal to	["\$a" -ge 0]
-lt	less than	["\$a" -lt 0]
-le	less than or equal to	["\$a" -le 10]

Important: Spaces are syntactically significant with the [command. Write ["\$number" -gt 0]; do not write ["\$number" -gt 0].

5.3 Number classification

```
if [ "$number" -gt 0 ]; then
    echo "The number is positive."
elif [ "$number" -lt 0 ]; then
    echo "The number is negative."
else
    echo "The number is zero."
fi
```

The conditions are checked from top to bottom. The first true branch runs; all remaining branches are skipped.

6 Arithmetic and Looping

6.1 Arithmetic expansion

Arithmetic expansion. The syntax `$( (expression) )`, which evaluates integer arithmetic and substitutes the result.

```
sum=$((sum + current))
digit=$((remaining % 10))
reverse=$((reverse * 10 + digit))
```

Operator	Operation	Operator	Operation
+	addition	-	subtraction
*	multiplication	/	integer division
%	remainder (modulo)	**	exponentiation

6.2 for loop

for loop. A loop that repeats a block once for every item in a generated or supplied list.

```
for variable in list
do
    commands
done
```

The first program generates the integers from 1 through 10:

```
for current in $(seq 1 10)
do
    sum=$((sum + current))
done
```

`seq 1 10` prints the required sequence. The command substitution `$(seq 1 10)` supplies that sequence to the loop.

6.3 Even and odd test

Dividing an even integer by 2 leaves a remainder of 0. Therefore:

```
if [ $((current % 2)) -eq 0 ]; then
    type="Even"
else
    type="Odd"
fi
```

6.4 while loop

while loop. A loop that continues as long as its condition returns a successful (true) status.

```
while [ condition ]
do
    commands
done
```

In the palindrome program, the loop continues until all decimal digits have been processed:

```
while [ "$remaining" -gt 0 ]
do
    digit=$((remaining % 10))
    reverse=$((reverse * 10 + digit))
    remaining=$((remaining / 10))
done
```

7 Input Validation with Bash Regular Expressions

Input validation. Checking input before processing it to confirm that it has the required format and value range.

Regular expression. A pattern used to test whether text follows a specified structure. In Bash, regular-expression matching is performed inside `[[ ]]` using the `=~` operator.


```
if [[ ! "$number" =~ ^[0-9]+$ ]]; then
    echo "Error: Please enter a non-negative whole number." >&2
    exit 1
fi
```

Pattern element	Meaning
[[...]]	Bash compound conditional expression
!	logical NOT; reverses the result of the following test
=~	matches the left-hand text against the right-hand pattern
^	beginning of the input
[0-9]	one decimal digit
+	one or more occurrences of the preceding pattern
\$	end of the input

The complete expression therefore accepts 0, 7, 121, and 00400, but rejects -3, 12.5, an empty input, and abc.

Important: The earlier pattern `^[0-9]$` accepts exactly one digit. The corrected pattern uses `+` so that a non-negative integer may contain one or more digits.

7.1 Decimal interpretation of leading zeros

```
original=$((10#$number))
```

The `10#` prefix asks Bash to interpret the input in base 10. This avoids treating a leading zero as an indication of another numeric base during arithmetic evaluation.

8 Palindrome Logic

Palindrome number. A number that reads identically from left to right and right to left, such as 7, 121, or 4554.

For `number = 123`, the algorithm works as follows:

Remaining before step	Digit extracted	Reverse after step	Remaining after step
123	3	3	12
12	2	32	1
1	1	321	0

The last step compares the original number with the reversed number:

```
if [ "$original" -eq "$reverse" ]; then
    echo "The number is a palindrome."
else
    echo "The number is not a palindrome."
fi
```

9 Command-Line Arguments and Positional Parameters

Command-line argument. A value written after the script name when the script is executed.

```
./file_line_counter.sh notes.txt
```

Parameter	Meaning
\$0	name used to start the script
\$1	first command-line argument
\$2	second command-line argument
\$#	total number of command-line arguments
@	all command-line arguments, preserved as separate values when quoted
\$\$	process identifier of the current shell
\$?	exit status of the most recently completed command

The file program requires exactly one path:

```
if [ "$#" -ne 1 ]; then
    echo "Usage: $0 FILE" >&2
    exit 1
fi

file_path="$1"
```

10 File-Test Operators

File-test operator. An operator used with [] to inspect a property of a filesystem path.

Operator	True when	Use in the program
-e	the path exists	reject a missing path
-f	the path is a regular file	reject a directory or special file
-r	the path is readable	reject a file the current user cannot read
-d	the path is a directory	useful in directory-processing scripts
-s	the file exists and is non-empty	useful when empty files are invalid
-x	the path is executable	useful when checking commands or scripts

10.1 Step-by-step path validation

The program validates the path step by step so that each error message is specific:

```
if [ ! -e "$file_path" ]; then
    echo "Error: '$file_path' does not exist." >&2
    exit 2
fi

if [ ! -f "$file_path" ]; then
    echo "Error: '$file_path' is not a regular file." >&2
    exit 3
fi

if [ ! -r "$file_path" ]; then
    echo "Error: '$file_path' is not readable." >&2
    exit 4
fi
```

11 Reading a File One Line at a Time

```
line_count=0

while IFS= read -r line || [ -n "$line" ]
do
    line_count=$((line_count + 1))
done < "$file_path"
```

Part	Purpose
line_count=0	initializes the counter before its first use
IFS=	prevents leading and trailing whitespace from being trimmed
read -r line	reads one line without interpreting backslashes
[-n "\$line"]	also counts a final non-empty line lacking a newline character
done < "\$file_path"	redirects the selected file into the entire loop

Input redirection. The operator < connects a file to the standard input of a command or compound command.

12 Command Substitution

Command substitution. The syntax \$(command), which runs a command and substitutes its standard output into the surrounding command.

```
echo "Current user: $(whoami)"
echo "Current directory: $(pwd)"
echo "Current date and time: $(date)"
echo "Host name: $(hostname)"
```

The four commands provide the following information:

Command	Result
whoami	login name of the effective current user
pwd	absolute path of the current working directory
date	current system date and time
hostname	host name assigned to the computer

13 The case Statement

case statement. A multi-way selection structure that compares one value with a series of patterns. It is often clearer than a long `if-elif` chain for menus and command dispatch.

13.1 General syntax

```
case "$value" in
  pattern_1)
    commands
    ;;
  pattern_2)
    commands
    ;;
  *)
    default_commands
    ;;
esac
```

- `case "$value" in` begins the selection.
- Each pattern ends with a closing parenthesis.
- `;;` terminates one branch and prevents execution from continuing into the following branch.
- `*` is the default pattern; it matches any value not matched earlier.
- `esac` is `case` written backward and closes the structure.

13.2 Menu dispatch

```
case "$choice" in
  1)
    echo "Current user: $(whoami)"
    ;;
  2)
    echo "Current directory: $(pwd)"
    ;;
  *)
    echo "Error: Invalid choice." >&2
    exit 1
    ;;
esac
```

14 Standard Output, Standard Error, and Exit Status

Standard output (stdout). The normal stream used by a program to display successful results. File descriptor 1 represents it.

Standard error (stderr). A separate stream intended for diagnostic and error messages. File descriptor 2 represents it.

```
echo "Normal result"  
echo "Error: Invalid input." >&2
```

>&2 redirects the output of `echo` to file descriptor 2. Separating normal results from errors helps when a script's normal output is redirected into another command or file.

Exit status. An integer returned by a program when it terminates. By convention, 0 means success and a non-zero value indicates an error.

```
exit 0  # successful completion  
exit 1  # general input or usage error
```

The exact non-zero values are chosen by the script author. Different values can identify different failures, as in the file-validation program.

15 Complete Corrected Program 1: Number Classification and Loop Processing

Suggested filename: number_loop.sh

Purpose: Demonstrates variables, input, numeric validation, if-elif-else, a for loop, arithmetic expansion, modulo, and a running total.

```

1  #!/usr/bin/env bash
2
3  number_1=100
4  echo "The number is $number_1"
5
6  read -r -p "Enter an integer: " number
7
8  if [[ ! "$number" =~ ^-?[0-9]+$ ]]; then
9      echo "Error: Please enter a valid integer." >&2
10     exit 1
11 fi
12
13 if [ "$number" -gt 0 ]; then
14     echo "The number is positive."
15 elif [ "$number" -lt 0 ]; then
16     echo "The number is negative."
17 else
18     echo "The number is zero."
19 fi
20
21 sum=0
22 echo "Processing numbers from 1 to 10"
23 echo "-----"
24
25 for current in $(seq 1 10)
26 do
27     sum=$((sum + current))
28
29     # Check whether the current number is even or odd.
30     if [ $((current % 2)) -eq 0 ]; then
31         type="Even"
32     else
33         type="Odd"
34     fi
35
36     echo "Number: $current, Type: $type, Running total: $sum"
37 done
38
39 echo "-----"
40 echo "The final sum is: $sum"
41
42 exit 0

```

Key execution result

The running total becomes $1 + 2 + \dots + 10 = 55$. The input number is classified independently of the loop. A separate variable named `current` is used so the loop does not overwrite the user's input stored in `number`.

16 Complete Corrected Program 2: Palindrome Number Checker

Suggested filename: palindrome_number.sh

Purpose: Demonstrates input validation, a `while` loop, decimal digit extraction, integer arithmetic, and equality comparison.

```

1  #!/usr/bin/env bash
2
3  # Take input from the user.
4  read -r -p "Enter a non-negative whole number: " number
5
6  # Validate the input.
7  if [[ ! "$number" =~ ^[0-9]+$ ]]; then
8      echo "Error: Please enter a non-negative whole number." >&2
9      exit 1
10 fi
11
12 original=$((10#$number))
13 remaining=$original
14 reverse=0
15
16 while [ "$remaining" -gt 0 ]
17 do
18     digit=$((remaining % 10))
19     reverse=$((reverse * 10 + digit))
20     remaining=$((remaining / 10))
21 done
22
23 echo "Original number: $original"
24 echo "Reversed number: $reverse"
25
26 # Finally, check whether the number is a palindrome.
27 if [ "$original" -eq "$reverse" ]; then
28     echo "The number is a palindrome."
29 else
30     echo "The number is not a palindrome."
31 fi
32
33 exit 0

```

Examples

Input	Reverse	Classification
121	121	palindrome
120	21	not a palindrome
0	0	palindrome

17 Complete Corrected Program 3: File Line Counter

Suggested filename: file_line_counter.sh

Purpose: Demonstrates positional parameters, argument-count validation, file tests, standard error, exit codes, input redirection, and safe line-by-line reading.

```
1  #!/usr/bin/env bash
2
3  # The script requires exactly one file path.
4  if [ "$#" -ne 1 ]; then
5      echo "Usage: $0 FILE" >&2
6      exit 1
7  fi
8
9  file_path="$1"
10
11 # Check whether the path exists.
12 if [ ! -e "$file_path" ]; then
13     echo "Error: '$file_path' does not exist." >&2
14     exit 2
15 fi
16
17 # Check whether the path identifies a regular file.
18 if [ ! -f "$file_path" ]; then
19     echo "Error: '$file_path' is not a regular file." >&2
20     exit 3
21 fi
22
23 # Check whether the current user can read the file.
24 if [ ! -r "$file_path" ]; then
25     echo "Error: '$file_path' is not readable." >&2
26     exit 4
27 fi
28
29 line_count=0
30
31 while IFS= read -r line || [ -n "$line" ]
32 do
33     line_count=$((line_count + 1))
34 done < "$file_path"
35
36 echo "File: $file_path"
37 echo "Status: The file exists, is regular, and is readable."
38 echo "Number of lines: $line_count"
39
40 exit 0
```

Example command

```
./file_line_counter.sh notes.txt
```


18 Complete Corrected Program 4: System Information Menu

Suggested filename: system_menu.sh

Purpose: Demonstrates menu input, a case statement, command substitution, a default branch, standard error, and exit status.

```

1  #!/usr/bin/env bash
2
3  echo "Select an option:"
4  echo "1. Display the current user"
5  echo "2. Display the current directory"
6  echo "3. Display the current date and time"
7  echo "4. Display the host name"
8
9  read -r -p "Enter your choice: " choice
10
11 case "$choice" in
12     1)
13         echo "Current user: $(whoami)"
14         ;;
15     2)
16         echo "Current directory: $(pwd)"
17         ;;
18     3)
19         echo "Current date and time: $(date)"
20         ;;
21     4)
22         echo "Host name: $(hostname)"
23         ;;
24     *)
25         echo "Error: Invalid choice." >&2
26         exit 1
27         ;;
28 esac
29
30 exit 0

```

19 Compact Syntax Reference

Construct	General syntax
Variable assignment	name=value
Variable expansion	"\$name" or "\${name}"
Read input	read -r -p "Prompt: " variable
Integer arithmetic	result=\$((expression))
Command substitution	result=\$(command)
Conditional	if [condition]; then commands; elif [condition]; then commands; else commands; fi
For loop	for item in list; do commands; done
While loop	while [condition]; do commands; done
Regular-expression test	[[text =~regex]]

Construct	General syntax
Case statement	<code>case "\$value" in pattern) commands ;; *) default ;; esac</code>
First argument	<code>\$1</code>
Argument count	<code>\$#</code>
File exists	<code>[-e "\$path"]</code>
Regular file	<code>[-f "\$path"]</code>
Readable file	<code>[-r "\$path"]</code>
Standard-error redirection	<code>echo "Error message" >&2</code>
Successful termination	<code>exit 0</code>

20 Common Errors and Their Corrections

Incorrect	Correct	Reason
<code>sum = 0</code>	<code>sum=0</code>	assignment cannot contain spaces around =
<code>[0-9]</code>	<code>[0-9]+</code>	+ permits one or more digits
counter used before initialization	<code>line_count=0</code>	initialize a counter before its first use
<code>[! -f \$file_path]</code>	<code>[! -f "\$file_path"]</code>	quoting preserves paths containing spaces
reuse number in the loop	use current in the loop	distinct names prevent accidental overwriting

21 Recommended Testing

Test normal, boundary, and invalid inputs rather than checking only one successful case.

Program	Normal or boundary tests	Invalid tests
Number loop	positive, negative, and zero	text, decimal, empty input
Palindrome	0, one digit, 121, 120, leading zeros	negative, decimal, letters
File counter	empty file, one line, final line without newline, path with spaces	no argument, missing path, directory, unreadable file
System menu	choices 1, 2, 3, and 4	0, 5, letters, empty input

22 Summary

The four programs collectively introduce the core control structures and data handling mechanisms needed for later operating-systems laboratory work:

- variables and quoted expansion store and retrieve data safely;
- `read` and positional parameters provide interactive and command-line input;
- `if` and `case` select actions;

- `for` and `while` repeat actions;
- arithmetic expansion supports counters, totals, and digit processing;
- regular expressions and file tests reject unsuitable input;
- redirection controls where input and diagnostic output flow; and
- exit codes communicate success or a particular failure to the calling environment.